CS-622 Homework 4
Thomas Rucinski
12/6/25

Dataset link: https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data

DATASET INFORMATION:

"Breast Cancer Wisconsin (Diagnostic) Data Set" is available on kaggle.com (link above) and is the dataset I used for my multi-layer perceptron classifier implementation.

The dataset contains a total of 30 measured and computed values **(input features)** from a digitized image of a fine needle aspirate (FNA) for 569 unique samples. FNA is a minimally-invasive biopsy technique which collects loose cells (not tissue). The image digitization for each FNA biopsies separates individual cells, and further image analysis is performed to measure the desired features.

The dataset is frequently used for model training and biological ML practice due to "clean" characteristics, such as no missing values, four significant figures per value, and confirmed diagnosis for all 569 samples. The dataset contains 32 columns, with column 1 being patient ID, and column 2 being patient diagnosis **(output feature).** The output is labeled as M or B (Malignant or Benign). The following 30 columns are the computed values for mean, standard error, and "worst" (mean of the three largest values) of the 10 features listed below in Figure 1 (a-j), respectively.

Ten real-valued features are computed for each cell nucleus:

a) radius (mean of distances from center to points on the perimeter)
b) texture (standard deviation of gray-scale values)
c) perimeter
d) area
e) smoothness (local variation in radius lengths)
f) compactness (perimeter^2 / area - 1.0)
g) concavity (severity of concave portions of the contour)
h) concave points (number of concave portions of the contour)
i) symmetry
j) fractal dimension ("coastline approximation" - 1)

The mean, standard error and "worst" or largest (mean of the three largest values) of these features were computed for each image, resulting in 30 features. For instance, field 3 is Mean Radius, field 13 is Radius SE, field 23 is Worst Radius.

Figure 1: Description of Measured Statistics Within Dataset
[**https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data**]

PREPROCESSING:

1. Data Loading and Dataframe Initialization: I loaded dataset using and used the hyperparameter "header=0" to remove the first row, which contained feature names.

2. Input and Output Dataframe Initialization: I initialized and loaded the input dataframe (30 features) and the output dataframe (1 feature).

3. Label Encoding: Then I used the LabelEncoder function from sk.learn.preprocessing to convert the output feature value from string ('M'/'B') to binary (1/0).

4. Dataset Splitting and Randomization: I split the datasets using train_test_split function from sklearn.model_selection, with test_size=0.2 and random_state=24.

5. Dataset Standardization: I scaled the input_train and input_test datasets using StandardScaler function from sk.learn.preprocessing.

MODEL VARIATION:

The assignment requires five different combinations of model parameters and subsequent evaluation metrics to be compared. I had trouble deciding on five, so I used a structured Grid Search with the following hyperparameters: 'hidden_layer_sizes', 'activation', 'solver', and a constant 'max_iter' of 2000 (see Figure 2). This Grid Search produces a total of twelve unique model combinations.

I constrained the architecture to the following:

- Hidden layer size of (100,) represents one hidden layer with 100 neurons.
- Hidden layer size (50, 50) represents two hidden layers with each layer containing 50 neurons.
- Hidden layer size (100, 50, 25) represents three hidden layers, with sizes of 100, 50, and 25 neurons respetively.

I constrained the activation function to be either:

- Rectified Linear Unit (relu), which is defined by having output between [0,∞].
- Hyperbolic Tangent (tanh), defined by having output between [-1,1]. These are both common activation functions used within machine learning algorithms.

For the solver, also known as learning strategy/optimizer, I constrained the options to:

- Adam (Adaptive moment estimation), which individualizes learning rate for each feature (generally faster, needs less optimization)

- SGD (Stochastic Gradient Descent), which fixes the learning rate across all features (generally slower, less likely to overfit).

In my output function, I list the twelve models with their respective hyperparameters in the following format: “Model n: ‘hidden_layer_sizes’ / ‘activation’ / ‘solver’”

```
# Defines the grid (models to be used for the performance evaluation)
# 3 x 2 x 2 x 1 = 12 unique models for comparison
param_grid = {
    'hidden_layer_sizes': [(100,), (50, 50), (100, 50, 25)],
    'activation': ['relu', 'tanh'],
    'solver': ['adam', 'sgd'],
    'max_iter': [2000]
```

Figure 2: Parameter Grid Setup for Experimental Models (12 Total Models Formed)

EVALUATION METRICS:

I created a get_metrics function which incorporated accuracy_score, f1_score, log_loss, and confusion_matrix functions from sklearn.metrics, and manually computed sensitivity and specificity (functions not in sklearn.metrics library) to provide comprehensive model performance. Figure 3 displays the full script output, and Table 2 displays the test dataset evaluation metrics for the 5 top-performing models based on accuracy.

```
Data Loaded Successfully.
Input Features: 30
Target Classes: ['B' 'M']

Running 12 MLP Classifier Variations:
Training MLP V1: (100,) / relu / adam
Training MLP V2: (100,) / relu / sgd
Training MLP V3: (50, 50) / relu / adam
Training MLP V4: (50, 50) / relu / sgd
Training MLP V5: (100, 50, 25) / relu / adam
Training MLP V6: (100, 50, 25) / relu / sgd
Training MLP V7: (100,) / tanh / adam
Training MLP V8: (100,) / tanh / sgd
Training MLP V9: (50, 50) / tanh / adam
Training MLP V10: (50, 50) / tanh / sgd
Training MLP V11: (100, 50, 25) / tanh / adam
Training MLP V12: (100, 50, 25) / tanh / sgd

TABLE 1: Training Data Metrics (top 5 models sorted by test rank)
 Rank                     Model Name  Train Accuracy  Train Sensitivity  Train Specificity  Train F1 Score  Train Log Loss
    1    MLP V8: (100,) / tanh / sgd        0.982418           0.964912           0.992958        0.976331        0.076470
    2 MLP V3: (50, 50) / relu / adam        1.000000           1.000000           1.000000        1.000000        0.003336
    2  MLP V4: (50, 50) / relu / sgd        0.982418           0.970760           0.989437        0.976471        0.061626
    2 MLP V9: (50, 50) / tanh / adam        1.000000           1.000000           1.000000        1.000000        0.005026
    2 MLP V10: (50, 50) / tanh / sgd        0.986813           0.976608           0.992958        0.982353        0.064739

TABLE 2: Test Data Metrics (top 5 models sorted by test rank)
 Rank                     Model Name  Test Accuracy  Test Sensitivity  Test Specificity  Test F1 Score  Test Log Loss
    1    MLP V8: (100,) / tanh / sgd       0.982456           0.95122          1.000000       0.975000       0.090144
    2 MLP V3: (50, 50) / relu / adam       0.973684           0.95122          0.986301       0.962963       0.094811
    2  MLP V4: (50, 50) / relu / sgd       0.973684           0.97561          0.972603       0.963855       0.080644
    2 MLP V9: (50, 50) / tanh / adam       0.973684           0.95122          0.986301       0.962963       0.114452
    2 MLP V10: (50, 50) / tanh / sgd       0.973684           0.95122          0.986301       0.962963       0.078942

Process finished with exit code 0
```

Figure 3: Full PyCharm Script Output Displaying MLP Variations and Evaluation Metrics

Table 1: Top 5 Multi-layer Perceptron Classifier Model Training Evaluation Metrics

Rank (Based on Test Accuracy)	Model Name	Accuracy	Sensitivity	Specificity	F1 Score	Log Loss
1	MLP V8	0.982418	0.964912	0.992958	0.976331	0.076470
2	MLP V3	1.000000	1.000000	1.000000	1.000000	0.003336
2	MLP V4	0.982418	0.970760	0.989437	0.976471	0.061626
2	MLP V9	1.000000	1.000000	1.000000	1.000000	0.005026
2	MLP V10	0.986813	0.976608	0.992958	0.982353	0.064739

Table 2: Top 5 Multi-layer Perceptron Classifier Model Test Evaluation Metrics

Rank (Based on Test Accuracy)	Model Name	Accuracy	Sensitivity	Specificity	F1 Score	Log Loss
1	MLP V8	0.982456	0.95122	1.000000	0.975000	0.090144
2	MLP V3	0.973684	0.95122	0.986301	0.962963	0.094811
2	MLP V4	0.973684	0.97561	0.972603	0.963855	0.080644
2	MLP V9	0.973684	0.95122	0.986301	0.962963	0.114452
2	MLP V10	0.973684	0.95122	0.986301	0.962963	0.078942

CONCLUSION:

Based on the Grid Search evaluation of 12 unique models, MLP V8 (hidden_layer_sizes=(100,),activation='tanh',solver='sgd') was identified as the optimal model for this dataset. MLP V8 achieved a test accuracy of ~98.25%, and a perfect test specificity of 1.000.

MLP V8 used a shallow architecture with a single hidden layer of 100 neurons. This suggests that the input features are distinct enough that multiple hidden layers were not necessary to achieve high overall model performance. It is notable that the models tied for 2nd in accuracy: MLP V3, MLP V4, MLP V9, and MLP V10, each have two hidden layers, which is somewhat contradictory for my conclusion of MLP V8's performance. It is also notable that the results are influenced by the random shuffling of the dataset in the preprocessing.

The activation function tanh outperforming relu for the top ranking (based on test accuracy). Tanh preserves negative values, unlike relu, allowing the model to effectively measure "below average" features as negative signals after standardization.

For the solver, while Adam is typically a faster optimizer, the SGD solver produced the highest generalizing model (based on test accuracy).

ADDITIONAL DETAILS:

While MLP V8 achieved the highest overall test accuracy, it presented a trade-off in clinical effectiveness. MLP V8's sensitivity was ~0.951, meaning around 5% of malignant samples were classified as benign.

In contrast, model MLP V4 ((50,50) / relu / sgd) achieved a higher test sensitivity of ~0.976. From a medical diagnostics perspective, this model would be preferred, because while the test specificity decreases from 1.000 to ~0.972 in MLP V4, which would produce slightly more false positives, the false negative rate (much more clinically significant in this case) would decrease in the model, and therefore be more successful in classifying malignancy from FNA biopsies.